Event Naming, Versioning, and notifications

Introduction

eP has evolved a lot and has seen lots of requirement change, leading to refactoring. In an event-sourced application, that poses a few problems. In order to promote stability and quicker delivery, then teams should follow proper guidance to making additive or substantial changes to events and follow a proper event naming and versioning convention.

Purpose

The purpose of this document is to provide guidance to all evicore team members who are producing and consuming events from eP. It has been observed teams are using different naming and versioning convention. For example, the version is postfix in the event name itself i.e. instead of RequestForServiceSubmitted teams are using RequestForServiceSubmittedV2. The intent of this document is to provide guidance on naming and versioning conventions (for example: not using the version in the event name and instead use Version attribute of event envelope) as per the current event/command envelope structure in eP and the current state of evicore.bus library, so that all the consumers of events are aware of changes been made in the events (additive or breaking) and are fully backward compatible (i.e. able to support previous version of events)

Message naming convention

In eP, there are two types of messages which are published to Kafka which can be classified as

- Commands
- Events

Commands are an instruction, telling a system to "do something" example Submit the request. Deny the authorization etc. Commands can be validated, approved/rejected, processed.

Events on the other hand reflect an immutable fact. Example: RequestSubmitted, Authorization approved. We can't approve/reject events in the past.

Naming commands

Commands typically have two parts:

- Verb

- Subject

The verb is usually the imperative mood, present tense. "Submit a Request". The verb is "submit", the mood is "imperative", and the subject, "Request". So the convention to follow can be

`<Verb><Subject>`

Naming Events

While commands are an imperative mood, events represent something that happened in the past, so we typically see events represented as a past-tense verb, along with a subject:

- Past-tense verb
- Subject

Similar to the commands, include both the verb and the subject as part of the message name, but reversed:

`<Subject><Verb>`

Note: Command and Event message should not have a version in their name i.e. RequestforservicesubmittedV2 is not a convention we want to follow

Event Versioning

What should require a new version?

A new version should be published whenever any of the following happens:

- An additive change of required attributes
- A subtraction of attributes
- An additive change of not required attributes
- In short, any change regardless of whether it is backwards compatible or not should result in a new version of an event. *This is to follow the versioning approach of AVRO.*

When should a new name instead of simply a new version be used?

- When there is a substantial change to the meaning or context of the event

How to version an event?

Every event should include a version number within the event itself. Event consumers can then use this version number and event name to determine how to handle/process the event. It is

recommended an event should follow [Sematic Versioning]() when versioning the event. Two part i.e. 1.1 and Four part 1.1.0.0 is also valid versioning scheme but should be avoided wherever possible.

Below JSON snippet is the example from the current eP production event where the event message has an attribute called **"Version"**. The team should utilize this "Version" attribute when it is appropriate to create a new version of an event.

Note: Payload attribute value has been decoded for illustration purpose


```
{
  "Payload": "{"Note":{"DocumentType":"notes","Key":"776afe8e-cda2-42de-a546-3adf701505e6","Metadata":{},"Creator":"epfertilityadvisor undefined","CreatorRole":"FertilityAdvisor","Content":"t","GeneratingAgent":"epfertilityadvisor@evicore.com","GeneratingAgentSource":"Add Note","GeneratingAgentContext":"FertilityAdvisor","DateCreated":"2021-03-16T15:39:22.7038599Z","ReferenceEntities":[{"Type":"requestForService","Key":"dde9a33e-eda8-47b9-80ba-05a7f0f94b83"}],"Context":"Care coordination note"}}",
  "Name": "NoteSaved",
  "Version": "1.0.0",
  "SerializationMethod": "JsonSerializationProvider",
  "Sent": "2021-03-16T15:39:22.7038599Z",
  "Key": "b3513257-2cfb-4f24-a7cc-d9a287adaf2e",
  "Metadata": {
    "referenceEntityKeys": {
      "requestForService": "dde9a33e-eda8-47b9-80ba-05a7f0f94b83",
      "notes": "776afe8e-cda2-42de-a546-3adf701505e6"
    },
    "eventType": "GeneralNotes",
    "generatingAgent": "epfertilityadvisor@evicore.com",
    "generatingAgentContext": null,
    "generatingSource": null,
    "generatingAgentRole": "FertilityAdvisor",
    "generatingAgentDisplayName": "epfertilityadvisor undefined",
    "summary": "User epfertilityadvisor undefined created a Note"
  }
}
```


In the above sample event message, there is a payload attribute that actually holds the event schema and if there is an addition of a not required field like "NoteAttachment" to the event payload minor version should be increased.


```
{
  "Payload": "{"Note":{"DocumentType":"notes","NoteAttachment": "", "Key":"776afe8e-cda2-42de-a546-3adf701505e6","Metadata":{},"Creator":"epfertilityadvisor undefined","CreatorRole":"FertilityAdvisor","Content":"t","GeneratingAgent":"epfertilityadvisor@evicore.com","GeneratingAgentSource":"Add Note","GeneratingAgentContext":"FertilityAdvisor","DateCreated":"2021-03-16T15:39:22.7038599Z","ReferenceEntities":[{"Type":"requestForService","Key":"dde9a33e-eda8-47b9-80ba-05a7f0f94b83"}],"Context":"Care coordination note"}}",
  "Name": "NoteSaved",
  "Version": "1.1.0",
```

```
  "SerializationMethod": "JsonSerializationProvider",
  "Sent": "2021-03-16T15:39:22.7038599Z",
  "Key": "b3513257-2cfb-4f24-a7cc-d9a287adaf2e",
  "Metadata": {
    "referenceEntityKeys": {
      "requestForService": "dde9a33e-eda8-47b9-80ba-05a7f0f94b83",
      "notes": "776afe8e-cda2-42de-a546-3adf701505e6"
    },
    "eventType": "GeneralNotes",
    "generatingAgent": "epfertilityadvisor@evicore.com",
    "generatingAgentContext": null,
    "generatingSource": null,
    "generatingAgentRole": "FertilityAdvisor",
    "generatingAgentDisplayName": "epfertilityadvisor undefined",
    "summary": "User epfertilityadvisor undefined created a Note"
  }
}
```


If there is a breaking change like adding a required attribute(s) a Major version should be changed. For example in the sample event message below a required attribute called “Description” is added


```
{
  "Payload": "{"Note":{"DocumentType":"notes","Description": "Test", "Key":"776afe8e-cda2-42de-a546-3adf701505e6","Metadata":{},"Creator":"epfertilityadvisor undefined","CreatorRole":"FertilityAdvisor","Content":"t","GeneratingAgent":"epfertilityadvisor@evicore.com","GeneratingAgentSource":"Add Note","GeneratingAgentContext":"FertilityAdvisor","DateCreated":"2021-03-16T15:39:22.7038599Z","ReferenceEntities":[{"Type":"requestForService","Key":"dde9a33e-eda8-47b9-80ba-05a7f0f94b83"}],"Context":"Care coordination note"}}",
  "Name": "NoteSaved",
  "Version": "2.0.0",
  "SerializationMethod": "JsonSerializationProvider",
  "Sent": "2021-03-16T15:39:22.7038599Z",
  "Key": "b3513257-2cfb-4f24-a7cc-d9a287adaf2e",
  "Metadata": {
    "referenceEntityKeys": {
      "requestForService": "dde9a33e-eda8-47b9-80ba-05a7f0f94b83",
      "notes": "776afe8e-cda2-42de-a546-3adf701505e6"
    },
    "eventType": "GeneralNotes",
    "generatingAgent": "epfertilityadvisor@evicore.com",
    "generatingAgentContext": null,
    "generatingSource": null,
    "generatingAgentRole": "FertilityAdvisor",
    "generatingAgentDisplayName": "epfertilityadvisor undefined",
    "summary": "User epfertilityadvisor undefined created a Note"
  }
}
```


Forward compatible

Generally, adding additional **not required** properties to an event (which is a minor version change) should not cause a versioning conflict with the event consumer. Meaning if we have a contract that defines the shape of our event, as long as we don’t break that contract, the

existing event consumers should be able to process our new event and ignore the additional properties.

If there is a change in the contract, then a new Major version of that event needs to be created and consumers should require specific changes to consume a new major version.

NOTES

New eviCore.Bus version 12.0.0 currently support forward compatibility

It's is recommended to upgrade to latest eviCore.Bus version 12.0.0 to support new event versioning strategy

Example of Forward Compatible event strategy

Consumer 1.0.0 Producer 1.0.0 = Consumed
Consumer 1.0.0 Producer 1.1.0 = Consumed!
Consumer 1.1.0 Producer 1.0.0 = Skipped
Consumer 1.0.0 Producer 2.0.0 = Skipped
Consumer 1.0.0.0 Producer 1.1.0.0 = Consumed
Consumer "Cat" Producer "Dog" = Skipped
Consumer "Frog" Producer "Frog" = Consumed! (Bus does not dictate a versioning scheme, but only supports exact matches for non Semantic Versions)
Consumer 1.0.0.0 Producer 1.1.0 = Consumed
Consumer 1.0.0. Producer 1.1.0.0 = Consumed
Consumer 1.0.0. Producer 1.1.0.alpha.beta.gamma = Consumed

Essentially, only the first two parts of the version are significant. As long as they are integers, Major == Major, and Consumer Minor <= Producer Minor, the message is Accepted.

Consumer subscribing to Service Message wrapper should honor the message version

Few of the microservices in eP subscribe to service message wrapper and there is a high probability for services processing the same event with the different version which can cause a side effect. It is required that every microservice should honor the event name and version in their implementation to have a predictable outcome.

How to publish an event?

If an event isn't forward compatible i.e. there is a contract change(Major version change), you might want to publish both v1 and v2 of the event that occurred. By this approach (double publish) it is made sure existing consumers continue using the old version of an event and giving them the opportunity to upgrade their consumer to listen to new events.

With double publish strategy, there should be a policy set that will allow deprecating the old version of an event once all consumers have upgraded to handle new version of an event

Note: Anyone using eviCore.Bus library need to make sure you update the version detail in your command/event message

```
    [Obsolete]
    [DataContract]
    public class SleepRequestForServiceSubmitted : EventMessage
    {
        /// <summary>
        /// The Request For Service that was saved. It is passed as a JObject to include extra properties but should conform to the <see cref="RequestForServiceDTO" />.
        /// </summary>
        [DataMember]
        public SleepRequestForServiceSubmittedEventModel Request { get; set; }

        [IgnoreDataMember]
        [JsonIgnore]
        public override ServiceMessageTypeDefinition ServiceMessageTypeDefinition { get; } =
            new ServiceMessageTypeDefinition("SleepRequestForServiceSubmitted",
            "1.0.1",
            new JsonSerializationProvider());
    }
```

and if there is a breaking change create a new class with the version and update the version in the service message definition as shown below

```
    [DataContract]
    public class SleepRequestForServiceSubmittedV2 : EventMessage
    {
        /// <summary>
        /// The Request For Service that was saved. It is passed as a JObject to include extra properties but should conform to the <see cref="RequestForServiceDTO" />.
        /// </summary>
        [DataMember]
        public SleepRequestForServiceSubmittedEventModelV2 Request { get; set; }

        [IgnoreDataMember]
        [JsonIgnore]
        public override ServiceMessageTypeDefinition ServiceMessageTypeDefinition { get; } =
            new ServiceMessageTypeDefinition("SleepRequestForServiceSubmitted",
            "2.0.0",
            new JsonSerializationProvider());
    }
```

How do we notify Event Changes?

- Email to ePJSONChanges@evicore.com and all scrum team with details about the changes and new schema prior to deploying in PROD
- Update the details of changes in a user story or feature

- Updates DevOps wiki or create a Confluence Page (till we get Kafka schema registry) to maintain event schema registry